



Magazin Online

Dan Lorena Elena
Calculatoare anul IV grupa II

Cuprins

1. **Introducere**
 - 1.1. **Descriere generală a proiectului**
 - 1.2. **Obiectivele proiectului**
 - 1.3. **Tehnologiile utilizate**
2. **Specificații**
 - 2.1. **Cerințe funcționale**
 - 2.2. **Cerințe non-funcționale**
3. **Documente de proiectare**
 - 3.1. **Arhitectura și aspecte tehnice**
 - 3.1.1. **Arhitectura**
 - 3.1.2. **Baza de Date**
 - 3.1.3. **Integrarea frontend-backend**
 - 3.1.4. **Securitate**
 - 3.1.5. **Dependency Injection**
 - 3.1.6. **Integrare JPA**
 - 3.2. **Diagrama Use Case**
 - 3.3. **Diagrama de clasă**
4. **Testarea**
 - 4.1. **Testarea aplicației**
 - 4.2. **Teste unitare**
 - 4.3. **Teste de integrare**
5. **Aplicația**
 - 5.1. **Baza de date**
 - 5.2. **Pagina de înregistrare cu Spring Security**
 - 5.3. **Pagina de acasă**
6. **Ce funcționează / Ce nu funcționează**
 - 6.1. **Funcționalități implementate**
 - 6.2. **Probleme întâmpinate**
7. **Surse**
 - 7.1. **Resurse utilizate**
 - 7.2. **Referințe externe**

1. Introducere

Proiectul "Magazin Online" este o aplicație web destinată pentru gestionarea și comercializarea produselor online. Scopul principal al aplicației este de a crea un sistem ușor de utilizat, unde utilizatorii pot vizualiza produsele disponibile, pot adăuga articole în coșul de cumpărături și pot efectua comenzi. De asemenea, aplicația permite administratorilor să gestioneze produsele și categoriile acestora.

Aplicația este construită pe baza unui model arhitectural tipic pentru aplicațiile moderne, având un frontend în HTML, CSS și JavaScript, iar pe partea de backend se folosește Spring Boot pentru a asigura funcționalitățile server-side. De asemenea, aplicația se conectează la o bază de date relațională pentru a stoca informațiile despre produse, utilizatori, categorii și comenzi.

2. Specificatii

2.1 Cerințe funcționale

Cerințele funcționale definesc comportamentul sistemului și sunt esențiale pentru a satisface nevoile utilizatorilor. Acestea se referă la ceea ce sistemul trebuie să facă, la modul în care interacționează cu utilizatorii și la felul în care răspunde la acțiunile acestora.

În cadrul acestui proiect, următoarele cerințe funcționale sunt îndeplinite:

1. Gestionarea produselor:

- Utilizatorii pot vizualiza o listă de produse disponibile în magazin, inclusiv numele, descrierea, prețul și stocul disponibil.
- Administrația poate adăuga, edita și șterge produse din aplicație.
- Fiecare produs este asociat cu o categorie specifică.
- Produsele au imagini atribuite care pot fi vizualizate de utilizatori.

2. Gestionarea categoriilor:

- Produsele sunt grupate în categorii (ex. "Mingi", "Accesorii", "Articole vestimentare", "Pantofi").
- Administrația poate adăuga, edita și șterge categorii de produse.
- Utilizatorii pot vizualiza produsele dintr-o categorie selectată.

3. Autentificare și autorizare:

- Utilizatorii trebuie să se autentifice pentru a putea plasa comenzi sau a modifica setările contului.
- Există un sistem de autentificare care permite utilizatorilor să se înregistreze, să se autentifice și să se deconecteze.
- Administratori și utilizatori au roluri diferite, iar administratorii pot accesa funcționalități suplimentare, cum ar fi gestionarea produselor și a categoriilor.

4. Coș de cumpărături:

- Utilizatorii pot adăuga produse în coșul de cumpărături și pot vedea detalii despre produsele adăugate (cantitate, preț total).
- Coșul de cumpărături poate fi actualizat (adăugare, eliminare produse).
- Utilizatorii pot plasa comenzi din coșul lor, finalizând achiziția.

5. Interfața utilizatorului:

- Aplicația oferă o interfață web responsivă, ușor de utilizat, care se adaptează la dispozitive mobile și desktop.
- Pagina principală afișează o listă de produse și categoriile acestora.
- Pagina de produse oferă informații detaliate despre fiecare produs, inclusiv descriere și preț.

6. Administrarea utilizatorilor:

- Administratorii pot gestiona utilizatorii (crea, edita, șterge conturi de utilizatori).
- Utilizatorii își pot actualiza informațiile de profil și parola.

7. Notificări și mesaje:

- Utilizatorii primesc notificări legate de starea comenzilor lor și de modificările aplicației.
- Administratorii pot trimite mesaje importante către utilizatori.

2.2 Cerințe non-funcționale

Cerințele non-funcționale definesc calitățile și constrângerile sistemului, cum ar fi performanța, scalabilitatea, securitatea și utilizabilitatea. Acestea sunt la fel de importante pentru succesul proiectului, întrucât influențează experiența utilizatorilor și eficiența aplicației.

1. Performanță:

- Aplicația trebuie să poată răspunde rapid la cererile utilizatorilor (în special pentru încărcarea paginilor de produse și categorii).
- Timpul de răspuns al aplicației nu trebuie să depășească 2 secunde pentru fiecare cerere.

2. Scalabilitate:

- Sistemul trebuie să fie capabil să se extindă ușor, astfel încât să poată gestiona un număr mare de utilizatori și produse.
- Baza de date trebuie să fie optimizată pentru a susține creșterea volumului de date.

3. Securitate:

- Aplicația trebuie să fie securizată pentru a preveni accesul neautorizat la informațiile sensibile (ex. parolele utilizatorilor și datele despre produse).
- Autentificarea utilizatorilor trebuie să fie realizată folosind standardele de securitate, cum ar fi criptarea parolelor.
- Accesul la anumite funcționalități (de exemplu, gestionarea produselor) trebuie să fie restricționat doar la utilizatorii cu rol de administrator.

4. Fiabilitate:

- Aplicația trebuie să fie disponibilă și să funcționeze corect în majoritatea situațiilor.
- Erorile trebuie gestionate corespunzător, iar utilizatorii trebuie să primească mesaje clare atunci când se produc erori.

5. Utilizabilitate:

- Interfața utilizatorului trebuie să fie intuitivă și ușor de folosit pentru utilizatorii de toate nivelele.
- Aplicația trebuie să aibă un design curat, care să permită utilizatorilor să navigheze ușor între pagini și să efectueze acțiuni fără a se simți confuzi.

6. Portabilitate:

- Aplicația trebuie să fie disponibilă pe diferite platforme, inclusiv pe dispozitive mobile (smartphone-uri și tablete), și să se adapteze la diferite rezoluții de ecran.
- Aplicația trebuie să funcționeze corect pe principalele browsere web (Chrome, Firefox, Safari, Edge).

7. Compatibilitate:

- Aplicația trebuie să fie compatibilă cu diferite versiuni ale Java și să poată fi rulată pe orice mediu de server compatibil cu Spring Boot.
- Baza de date trebuie să poată fi exportată și importată între diferite medii de dezvoltare și producție.

3. Documente de proiectare

3.1 Arhitectura și aspecte tehnice

3.1.1 Arhitectura

Arhitectura aplicației este bazată pe un model **MVC** (Model-View-Controller), utilizând **Spring Boot** pentru backend și **HTML**, **CSS** și **JavaScript** pentru frontend. Acest model asigură separarea clară a responsabilităților și facilitează gestionarea codului într-o aplicație scalabilă.

- **Model:** Reprezintă datele și logica aplicației. Aici avem entitățile de bază (ex. **Category**, **Product**, **User**), care sunt gestionate de Spring Data JPA pentru a interacționa cu baza de date.
- **View:** Este partea de interfață cu utilizatorul, construită folosind **Thymeleaf** pentru a genera pagini HTML dinamice.
- **Controller:** Gestionarea logicii de business și manipularea cererilor HTTP. Spring MVC folosește controale pentru a prelua cererile și a răspunde cu date din model sau vizualizări din view.

Aplicația utilizează tehnologia **Spring Boot** pentru a construi un serviciu web rapid și scalabil, iar frontend-ul este responsiv, oferind o experiență prietenoasă pentru utilizatori, indiferent de dispozitivul folosit.

3.1.2 Baza de date

Aplicația utilizează **MySQL** pentru stocarea datelor. Baza de date conține următoarele entități principale:

- **Category:** Reprezintă categoriile de produse, fiecare categorie având un nume și o descriere.
- **Product:** Reprezintă produsele din magazin, cu attribute precum nume, descriere, preț, stoc, și imagine asociată.
- **User:** Reprezintă utilizatorii aplicației, cu informații despre datele de autentificare, roluri și preferințe.

Fiecare categorie poate conține mai multe produse, iar un produs este asociat cu o singură categorie. De asemenea, sunt folosite relații **OneToMany** și **ManyToOne** pentru a asigura integritatea referențială a datelor

Tabelele din baza de date:

1. Tabela categories

Această tabelă stochează informații despre categoriile de produse disponibile în magazinul online.

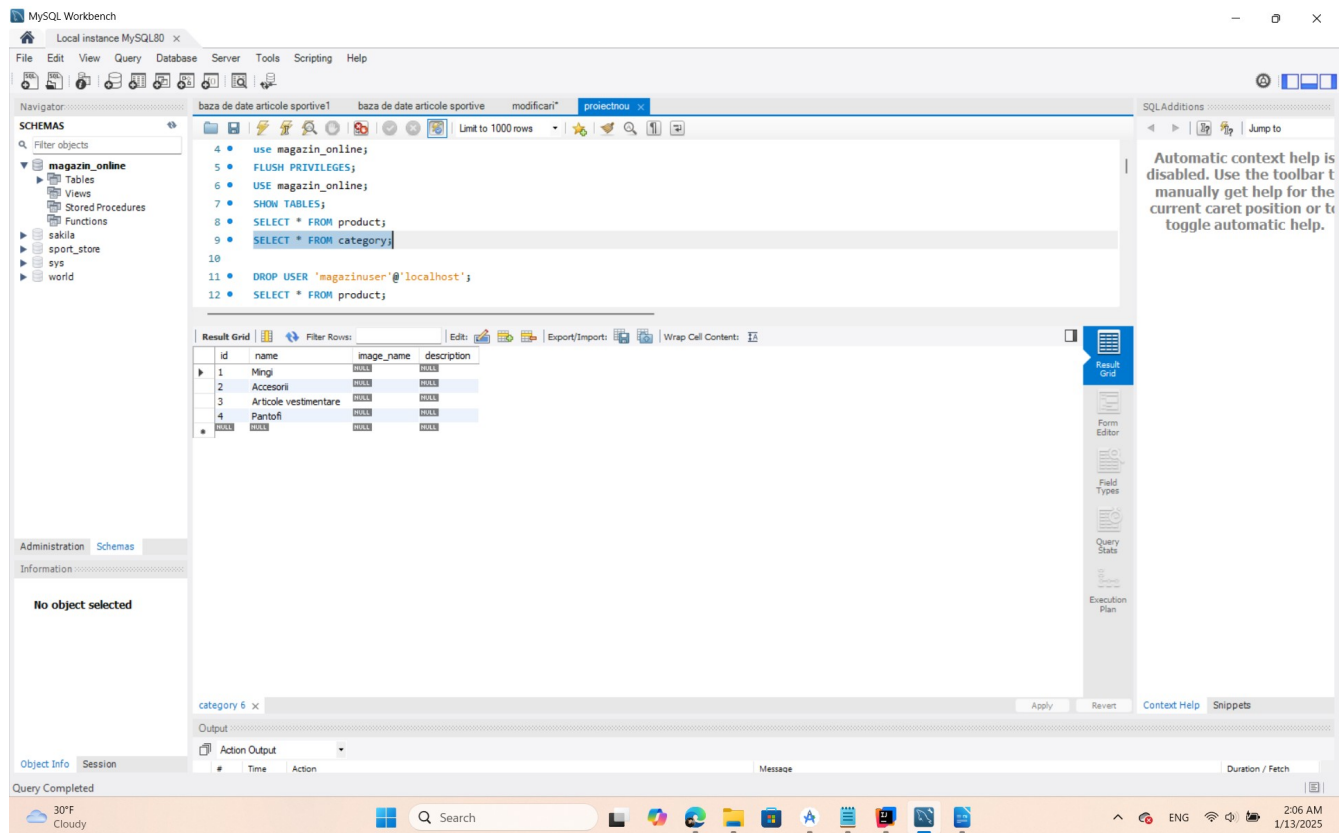
Coloane:

- **id**: Identificator unic al categoriei, de tip **LONG** și **PRIMARY KEY**.
- **name**: Numele categoriei, de tip **VARCHAR(255)**.
- **description**: Descrierea categoriei, de tip **TEXT**.

Relații:

- Un **produs** este asociat cu o **categorie**. Astfel, există o relație **OneToMany** între **Category** și **Product**, adică o categorie poate conține mai multe produse

Această tabelă conține 4 categorii: Mingi, Accesorii, Articole Vestimentare, Pantofi. Fiecarei categorii îi sunt atribuite 5 produse.



2. Tabela products

Această tabelă stochează informații despre produsele care sunt disponibile în fiecare categorie din magazinul online.

Coloane:

- **id**: Identificator unic al produsului, de tip **LONG** și **PRIMARY KEY**.
- **name**: Numele produsului, de tip **VARCHAR(255)**.
- **description**: Descrierea produsului, de tip **TEXT**.
- **price**: Prețul produsului, de tip **DECIMAL(10, 2)**.
- **stock**: Cantitatea disponibilă în stoc, de tip **INTEGER**.
- **category_id**: Cheia străină care face legătura între produs și categorie, de tip **BIGINT**.

Relații:

- Fiecare **produs** este asociat cu o **categorie** (relația **ManyToOne**). Un produs este legat de o categorie specifică prin câmpul **category_id**.
- De asemenea, relația de tip **OneToMany** între **Category** și **Product** este reflectată în modelul de date al aplicației, unde o categorie poate conține mai multe produse

In tabela **Products** avem 20 de produse care sunt impartite cate 5 pentru fiecare categorie.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```

1 CREATE DATABASE magazin_online;
2 CREATE USER 'magazinuser'@'localhost' IDENTIFIED BY 'password';
3 GRANT ALL PRIVILEGES ON magazin_online.* TO 'magazinuser'@'localhost';
4 use magazin_online;
5 FLUSH PRIVILEGES;
6 USE magazin_online;
7 SHOW TABLES;
8 SELECT * FROM product;
9 SELECT * FROM category;

```

The Results Grid shows the output of the query, displaying a list of products with columns: id, description, name, price, stock, category_id, and image. The data is as follows:

id	description	name	price	stock	category_id	image
1	Minge fotbal Nike Park Team, marime 5, Alb/Neg...	Minge de fotbal Nike	350.5	100	1	produse/produs1.jpg
2	Prodicator: Spalding Categori: Minge baschet ...	Mingea de baschet Spalding	200	50	1	produse/produs2.jpg
3	Mingea de tenis WILSON Roland Garros Jumbo ...	Minge de tenis Wilson	150	200	1	produse/produs3.jpg
4	Minge volei marca Mikasa. Confectionata din 8 p...	Minge de volei Mikasa	80	150	1	produse/produs4.jpg
5	Un upgrade al popularului clasic Barbarian Matc...	Minge de rugby Gilbert	300	75	1	produse/produs5.jpg
6	Geantă încălțătoare și rezistentă	Geantă pentru echipament sportiv	250	20	2	produse/produs6.jpg
7	Benitțe pentru frunte, set de 3	Benitțe absorbante	30	15	2	produse/produs7.jpg
8	Sticlă de hidratare de 1L	Sticlă de apă Nike	60	50	2	produse/produs8.jpg
9	Support pentru depozitarea mingilor	Support pentru mingi	150	20	2	produse/produs9.jpg
10	Ochelari pentru competiții de înot	Ochelari de înot Speedo	125	35	2	produse/produs10.jpg
11	Tricou ușor, material respirabil	Tricou sport Nike	225	200	3	produse/produs11.jpg
12	Pantaloni lungi pentru fitness	Pantaloni de antrenament Adidas	100	150	3	produse/produs12.jpg
13	Șapcă din material ușor	Șapcă sport Puma	70	40	3	produse/produs13.jpg
14	Honorac pentru vreme rece	Honorac sportiv Reebok	250	40	3	produse/produs14.jpg
15	Jachetă ușoară pentru outdoor	Jachetă impermeabilă Columbi	400	20	3	produse/produs15.jpg
16	Pantofi ușori și confortabili	Adidas pentru alergare Nike	450	100	4	produse/produs16.jpg
17	Pantofi special concepuți pentru baschet	Pantofi de baschet Air Jordan	600	15	4	produse/produs17.jpg
18	Pantofi rezistenți la apă	Pantofi pentru drumeții Salomon	550	10	4	produse/produs18.jpg
19	Sandale ideale pentru vara	Sandale sport Columbia	200	35	4	produse/produs19.jpg
20	Papuci pentru piscină și duș	Papuci sport Adidas	80	40	4	produse/produs20.jpg

3. Tabela users

Această tabelă stochează informații despre utilizatori, cum ar fi datele de autentificare și rolurile acestora (ex. **Admin**, **User**).

Coloane:

- **id**: Identificator unic al utilizatorului, de tip **LONG** și **PRIMARY KEY**.
- **username**: Numele de utilizator pentru autentificare, de tip **VARCHAR(255)**.
- **password**: Parola utilizatorului, de tip **VARCHAR(255)** (acesta va fi criptată).
- **role**: Rolul utilizatorului, de tip **VARCHAR(50)** (ex. ADMIN, USER).

4. Tabela **orders**

Această tabelă stochează informațiile generale ale unei comenzi, cum ar fi utilizatorul care a plasat comanda, data comenzii, și statusul acesteia.

Coloane:

- **id**: Identificator unic al comenzii, de tip **LONG** și **PRIMARY KEY**.
- **order_date**: Data la care a fost plasată comanda, de tip **TIMESTAMP**.
- **status**: Statusul comenzii (ex: PENDING, COMPLETED, CANCELLED), de tip **VARCHAR(50)**.

5. Tabela **OrderDetail**

Tabela **OrderDetail** poate fi definită pentru a stoca informații detaliate pentru fiecare produs dintr-o comandă. Se va lega de tabela **orders** și **products** pentru a crea o relație între comenzi și produsele comandate.

Coloane:

1. **id**: Identificatorul unic al detaliului de comandă, de tip **LONG**.
2. **quantity**: Cantitatea produsului comandat, de tip **INT**.
3. **price**: Prețul produsului la momentul comenzii, de tip **DOUBLE**.

Cum funcționează aplicația?

1. Gestionarea categoriilor:

- Administratorii pot adăuga, edita sau șterge categorii din aplicație.
- Fiecare categorie va conține o listă de produse asociate.

2. Gestionarea produselor:

- Administratorii pot adăuga, edita sau șterge produse în funcție de categoria acestora.
- Fiecare produs are un preț, o descriere și o cantitate în stoc.

3. Autentificare utilizatori:

- Utilizatorii se pot înregistra și autentifica pentru a accesa produsele și pentru a efectua comenzi.
- Datele de autentificare sunt criptate în baza de date, iar rolurile sunt folosite pentru a restricționa accesul la anumite funcționalități.

3.1.3 Integrarea frontend-backend

Frontend-ul aplicației este realizat folosind **Thymeleaf** ca engine de template, care permite crearea de pagini web dinamice. Backend-ul este implementat cu Spring Boot, iar datele sunt transmise între cele două prin intermediul controller-elor Spring.

Comunicarea între frontend și backend se face prin intermediul request-urilor HTTP (GET, POST, DELETE), iar datele sunt returnate de server în format HTML pentru a fi afișate în browserul utilizatorului. De exemplu, atunci când utilizatorul accesează o categorie, backend-ul preia produsele din acea categorie și le trimite la view-ul Thymeleaf.

3.1.4 Securitate

Aplicația utilizează **Spring Security** pentru autentificarea și autorizarea utilizatorilor. Înregistrarea și autentificarea se fac folosind un formular simplu, iar utilizatorii pot avea roluri diferite: **Admin** sau **User**.

Pentru a proteja resursele sensibile și a controla accesul, se implementează următoarele măsuri:

- **Autentificare:** Parolele utilizatorilor sunt criptate folosind algoritmi de criptare precum **BCrypt**.
- **Autorizare:** Accesul la anumite funcționalități este restricționat în funcție de rolul utilizatorului. De exemplu, doar administratorii pot adăuga sau modifica produse.

3.1.5 Dependency Injection

Spring Boot utilizează **Dependency Injection** (DI) pentru a gestiona instanțierea și injectarea dependențelor în clasele componente ale aplicației. Acest lucru permite o gestionare mai ușoară a serviciilor și componentelor, reducând legăturile între clase și facilitând testarea unităților.

Exemplu de injectare a dependențelor:

```
@Autowired
```

```
private CategoryService categoryService;
```

```
@Autowired
```

```
private ProductService productService;
```

3.1.6 Integrarea JPA

Spring Data JPA este utilizat pentru gestionarea interacțiunii cu baza de date. Repository-urile sunt create folosind interfețe care extind **JpaRepository**, iar Spring Boot le va implementa automat. Aceasta permite realizarea de operațiuni CRUD (Create, Read, Update, Delete) într-un mod eficient și fără a fi nevoie de scrierea de cod SQL manual.

Exemplu de repository:

```
public interface CategoryRepository extends JpaRepository<Category, Long> {

    @Query("SELECT MIN(c.id) FROM Category c GROUP BY c.name")
    List<Long> findIdsToKeep();

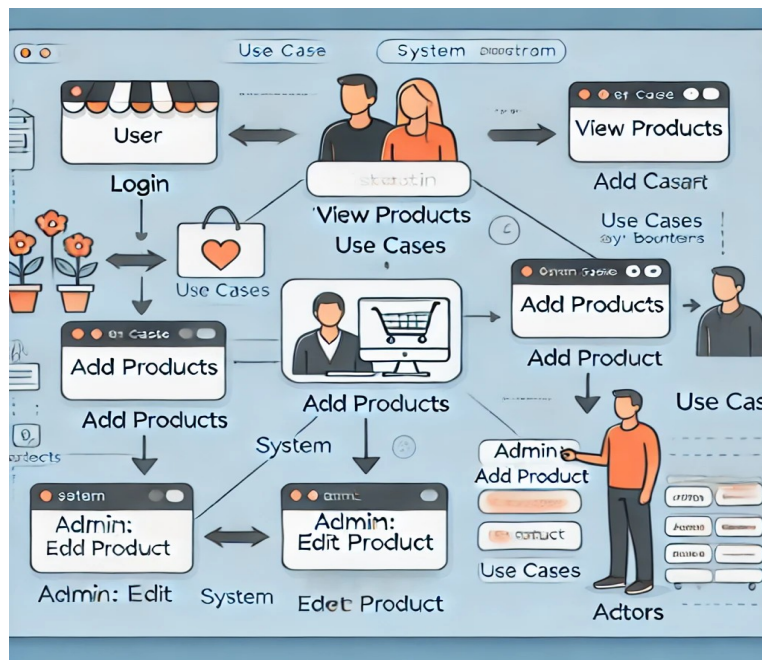
    void deleteByIdNotIn(List<Long> idsToKeep);
    @Modifying
    @Query("DELETE FROM Category c WHERE c.id NOT IN (" +
        "SELECT MIN(c2.id) FROM Category c2 GROUP BY c2.name)")
    void deleteDuplicateCategories();
}
```

3.2 Diagrama Use Case

Diagrama Use Case ilustrează interacțiunile dintre utilizatori și sistem. Acesta este un instrument de modelare a cerințelor funcționale ale aplicației și ajută la înțelegerea modului în care utilizatorii vor interacționa cu sistemul.

Exemplu de Use Case:

- **Autentificare utilizator:** Utilizatorul se autentifică pentru a putea plasa comenzi.
- **Vizualizare produse:** Utilizatorul poate vizualiza produsele disponibile și poate adăuga produse în coșul de cumpărături.
- **Administrează produse:** Administratorul poate adăuga, edita și șterge produse.



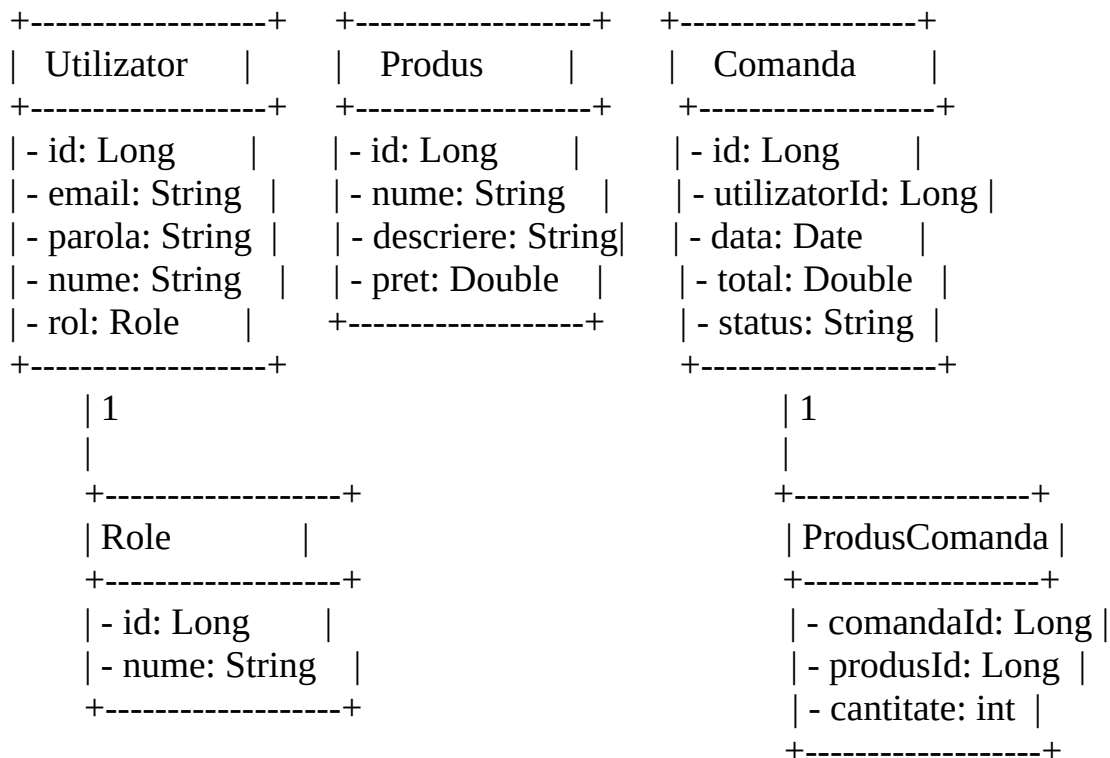
3.3. Diagrama de clasă

Diagrama de clasă este un instrument esențial în dezvoltarea aplicațiilor orientate pe obiecte. Aceasta ajută la vizualizarea structurii aplicației, arătând clasele, relațiile dintre ele și interacțiunile dintre obiecte. În cadrul acestui proiect, diagrama de clasă va reflecta entitățile aplicației și modul în care acestea sunt conectate între ele.

În cadrul aplicației **Magazin Sportiv**, vom avea mai multe clase care reprezintă diverse entități din baza de date și logica de business. De exemplu:

- **Utilizator** – Reprezintă un utilizator al magazinului (inclusiv detalii de autentificare).
- **Produs** – Reprezintă un produs din magazin, incluzând informații precum numele, prețul și descrierea produsului.
- **Comanda** – Reprezintă o comandă plasată de un utilizator, incluzând produsele comandate și starea comenzii.
- **Role** – Clasele care definesc rolurile utilizatorilor (ex. Admin, Client).

Diagrama de clasă ar trebui să arate relațiile între aceste entități, incluzând asocieri, moșteniri și dependențe între clase.



Explicații

- **Utilizator:** Clasa `Utilizator` va conține detalii legate de utilizatori, cum ar fi email-ul, parola criptată și numele complet. De asemenea, va include un câmp care referă la rolul utilizatorului (ex. admin, client).
- **Produs:** Clasa `Produs` va reprezenta un produs disponibil în magazin, incluzând informații precum numele, descrierea și prețul.
- **Comanda:** Clasa `Comanda` reprezintă o comandă plasată de un utilizator. Fiecare comandă poate conține mai multe produse, iar câmpul `total` va stoca suma totală a comenzii.
- **Role:** Clasa `Role` va defini rolurile posibile ale utilizatorilor (Admin, Client, etc.), iar relația dintre `Utilizator` și `Role` este de tipul **many-to-one**, adică un utilizator are un singur rol, dar un rol poate fi asociat cu mai mulți utilizatori.
- **ProdusComanda:** Clasa de legătură între `Comanda` și `Produs`, deoarece o comandă poate conține mai multe produse, iar fiecare produs poate fi parte din mai multe comenzi.

4. Testarea aplicației

Testarea este o parte esențială a procesului de dezvoltare software, asigurându-se că aplicația funcționează corect și conform așteptărilor. În acest capitol, vom discuta despre procesul de testare al aplicației **Magazin Sportiv**, inclusiv testele unitare și testele de integrare.

4.1. Testarea aplicației

Testarea aplicației include verificarea funcționalității acesteia prin diverse tipuri de teste. Aceste teste pot fi efectuate pe componentele individuale (teste unitare) sau pe întreaga aplicație (teste de integrare). În plus, este important ca aplicația să fie testată și în condiții de utilizare reală pentru a asigura o experiență de utilizator fără erori.

În cadrul acestui proiect, testarea va include următoarele tipuri de teste:

- **Teste unitare:** Acestea testează componentele individuale ale aplicației, cum ar fi metodele din clase sau funcționalitățile din cadrul serviciilor.
- **Teste de integrare:** Aceste teste verifică interacțiunile dintre diferitele componente ale aplicației, cum ar fi integrarea între backend și frontend sau comunicarea dintre aplicație și baza de date.

4.2. Teste unitare

Testele unitare sunt folosite pentru a verifica dacă o unitate mică de cod (de obicei o metodă) funcționează corect. În aplicația **Magazin Online**, putem folosi testele unitare pentru a verifica funcționalitățile implementate în backend, cum ar fi autentificarea utilizatorilor, manipularea comenzilor și interacțiunea cu baza de date.

Exemple de teste unitare pentru aplicația noastră:

1. Testarea autentificării utilizatorilor:

- Verificarea că un utilizator existent poate fi autentificat cu succes.
- Verificarea că un utilizator inexistent nu poate fi autentificat.
- Verificarea că o parolă incorectă nu permite autentificarea.

2. Testarea operațiunilor cu produsele:

- Verificarea că produsele pot fi adăugate corect în baza de date.
- Verificarea că produsele pot fi editate corect.
- Verificarea că produsele pot fi șterse din baza de date.

4.3. Teste de integrare

Teste de integrare sunt utilizate pentru a verifica interacțiunile între diferitele module ale aplicației, de obicei între aplicația backend și baza de date, dar și între backend și frontend. Aceste teste sunt esențiale pentru a asigura că fluxul de date între componenta de autentificare, stocarea datelor și răspunsurile către utilizator sunt corecte.

Exemple de teste de integrare pentru aplicația noastră:

1. Testarea comunicării între backend și frontend:

- Verificarea că datele de autentificare trimise de frontend sunt corect preluate de backend.
- Verificarea că backend-ul returnează un răspuns corect și că acesta este procesat corespunzător în frontend.

2. Testarea comunicării între aplicație și baza de date:

- Verificarea că un utilizator nou poate fi adăugat în baza de date și că datele sunt persistente.
- Verificarea că o comandă plasată de un utilizator este salvată corect în baza de date.

Concluzii privind testarea

Testarea aplicației **Magazin Online** este esențială pentru asigurarea unei aplicații stabile și sigure. Testele unitare ne ajută să verificăm logică internă a componentelor individuale, iar testele de integrare ne ajută să verificăm interacțiunile dintre diferitele părți ale aplicației, inclusiv comunicarea dintre backend și frontend și accesul la baza de date.

Testele vor fi esențiale pentru a garanta că aplicația funcționează corect în diverse scenarii, oferind utilizatorilor o experiență sigură și fluidă.

5. Aplicația

Aplicația **Magazin Online** este construită pentru a oferi o experiență completă de utilizator pentru gestionarea unui magazin online de produse sportive. Aceasta include interfețele pentru înregistrare și autentificare, gestionarea produselor, plasarea comenzilor și vizualizarea acestora. În această secțiune, vom discuta despre componentele principale ale aplicației, cum ar fi **baza de date**, **pagina de înregistrare cu Spring Security**, și **pagina de acasă**.

5.1. Baza de date

În aplicația **Magazin Online**, baza de date joacă un rol esențial în stocarea informațiilor despre utilizatori, produse, comenzi și alte entități relevante. În acest proiect, baza de date este construită folosind **JPA (Java Persistence API)** pentru a interacționa cu o bază de date relațională, iar **Spring Data JPA** este utilizat pentru a facilita interogările și operațiile CRUD (Create, Read, Update, Delete).

Baza de date include următoarele entități principale:

- **User**: Stocază informațiile despre utilizatori, inclusiv numele, email-ul și parola criptată. Este folosită pentru autentificare și autorizare.
- **Produsct**: Reprezintă produsele disponibile în magazin. Include informații precum numele produsului, descrierea, prețul și stocul.
- **Orders**: Reprezintă comenzile plasate de utilizatori. Include detalii despre produsele comandate, cantitățile și statusul comenzii.

În cadrul aplicației, aceste entități sunt gestionate prin intermediul **Spring Data JPA**, care se ocupă de persistarea datelor și interogările bazei de date.

The screenshot shows the MySQL Workbench interface. The top toolbar includes icons for File, Edit, View, Query, Database, Server, Tools, Scripting, and Help. The left sidebar shows the 'SCHEMAS' panel with a tree view containing 'magazin_online' (Tables, Views, Stored Procedures, Functions), 'sakila', 'sport_store', 'sys', and 'world'. The main editor displays a SQL script for creating a database, user, and tables, followed by a query to select all data from the 'product' and 'category' tables. The 'Result Grid' at the bottom shows the output of the query, displaying columns: id, description, name, price, stock, category_id, and image. The data includes various sports equipment like basketballs, tennis rackets, and shoes. The right sidebar contains a 'SQL Additions' panel with a message about automatic context help being disabled. The bottom status bar shows 'Query Completed', '30°F Cloudy', and the system clock '1:55 AM 1/13/2025'.

```

1 • CREATE DATABASE magazin_online;
2 • CREATE USER 'magazinuser'@'localhost' IDENTIFIED BY 'password';
3 • GRANT ALL PRIVILEGES ON magazin_online.* TO 'magazinuser'@'localhost';
4 • use magazin_online;
5 • FLUSH PRIVILEGES;
6 • USE magazin_online;
7 • SHOW TABLES;
8 • SELECT * FROM product;
9 • SELECT * FROM category;

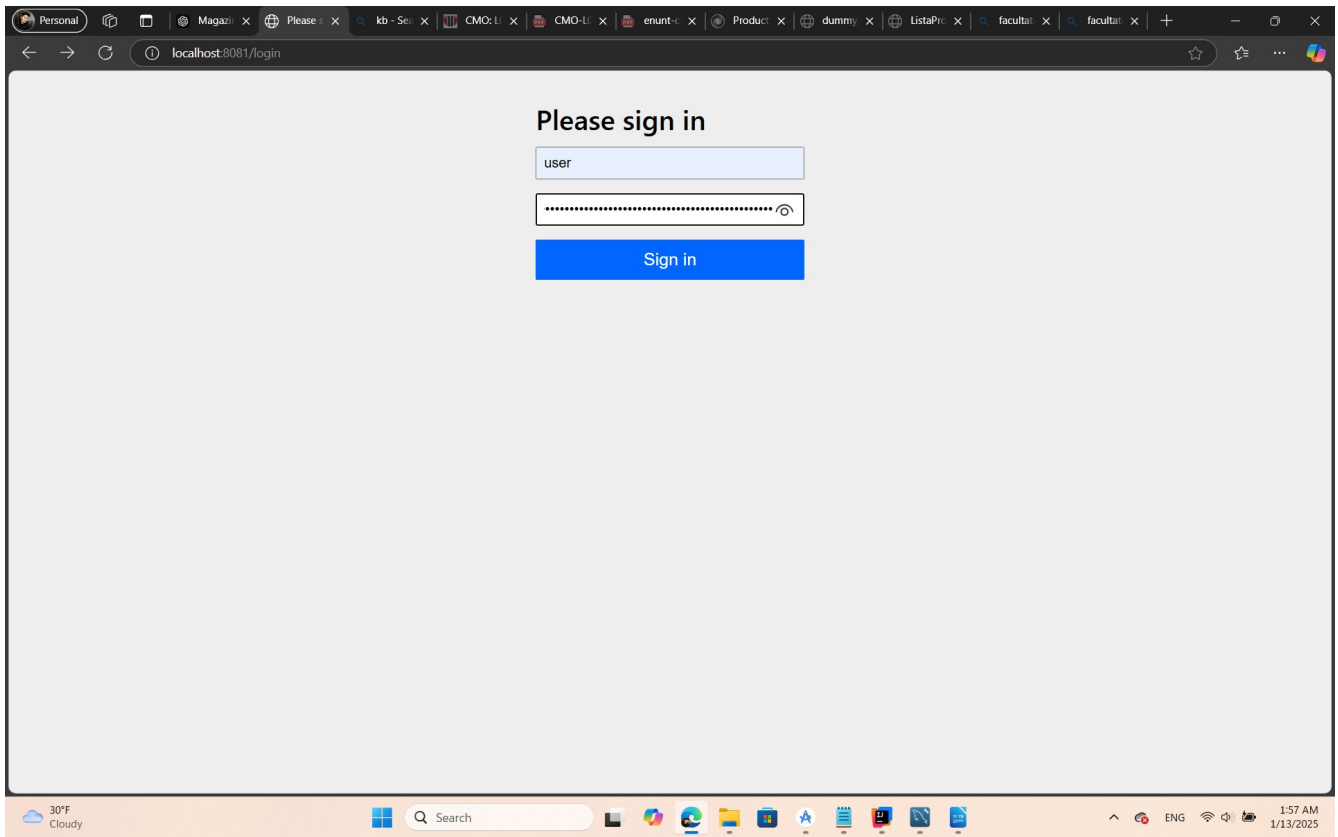
```

id	description	name	price	stock	category_id	image
1	Minge fotbal Nike Park Team, marime 5, Alb/Neg...	Minge de fotbal Nike	350.5	100	1	produse/produs1.jpg
2	Produsator: Spalding Categoria: Minge baschet...	Mingea de baschet Spalding	200	50	1	produse/produs2.jpg
3	Mingea de tenis WILSON Roland Garros Jumbo...	Minge de tenis Wilson	150	200	1	produse/produs3.jpg
4	Minge volei marca Mikasa. Confectionata din 8 p...	Minge de volei Mikasa	80	150	1	produse/produs4.jpg
5	Un upgrade al popularului clasic Barbarian Matc...	Minge de rugby Gilbert	300	75	1	produse/produs5.jpg
6	Geantă încălțătoare și rezistentă	Geantă pentru echipament sportiv	250	20	2	produse/produs6.jpg
7	Bentite pentru frunte, set de 3	Bentite absorbante	30	15	2	produse/produs7.jpg
8	Sticlă de hidratare de 1L	Sticlă de apă Nike	60	50	2	produse/produs8.jpg
9	Suport pentru depozitarea mingilor	Suport pentru mingi	150	20	2	produse/produs9.jpg
10	Ochelari pentru competiții de înot	Ochelari de înot Speedo	125	35	2	produse/produs10.jpg
11	Tricou usor, material respirabil	Tricou sport Nike	225	200	3	produse/produs11.jpg
12	Pantaloni lungi pentru fitness	Pantaloni de antrenament Adidas	100	150	3	produse/produs12.jpg
13	Sapcă din material usor	Sapcă sport Puma	70	40	3	produse/produs13.jpg
14	Honorac pentru vreme rece	Honorac sportiv Reebok	250	40	3	produse/produs14.jpg
15	Jachetă usoră pentru outdoor	Jachetă impermeabilă Columbi	400	20	3	produse/produs15.jpg
16	Pantofi usori și confortabili	Adidas pentru alergare Nike	450	100	4	produse/produs16.jpg
17	Pantofi special concepuți pentru baschet	Pantofi de baschet Air Jordan	600	15	4	produse/produs17.jpg
18	Pantofi rezistenți la apă	Pantofi pentru drumetii Salomon	550	10	4	produse/produs18.jpg
19	Sandale ideale pentru vara	Sandale sport Columbia	200	35	4	produse/produs19.jpg
20	Papuci pentru picioare și duș	Papuci sport Adidas	80	40	4	produse/produs20.jpg

5.2. Pagina de înregistrare cu Spring Security

Pagina de înregistrare este o componentă esențială a aplicației **Magazin Online**. Aceasta permite utilizatorilor să își creeze un cont, să își seteze parola și să se autentifice în aplicație. În această pagină, **Spring Security** este utilizat pentru a securiza operațiunile de autentificare și autorizare.

La înregistrare, utilizatorii trebuie să furnizeze informațiile necesare, cum ar fi numele, email-ul și parola. Parola este criptată folosind **BCryptPasswordEncoder** înainte de a fi stocată în baza de date. După înregistrare, utilizatorul poate accesa pagina de autentificare pentru a se conecta.



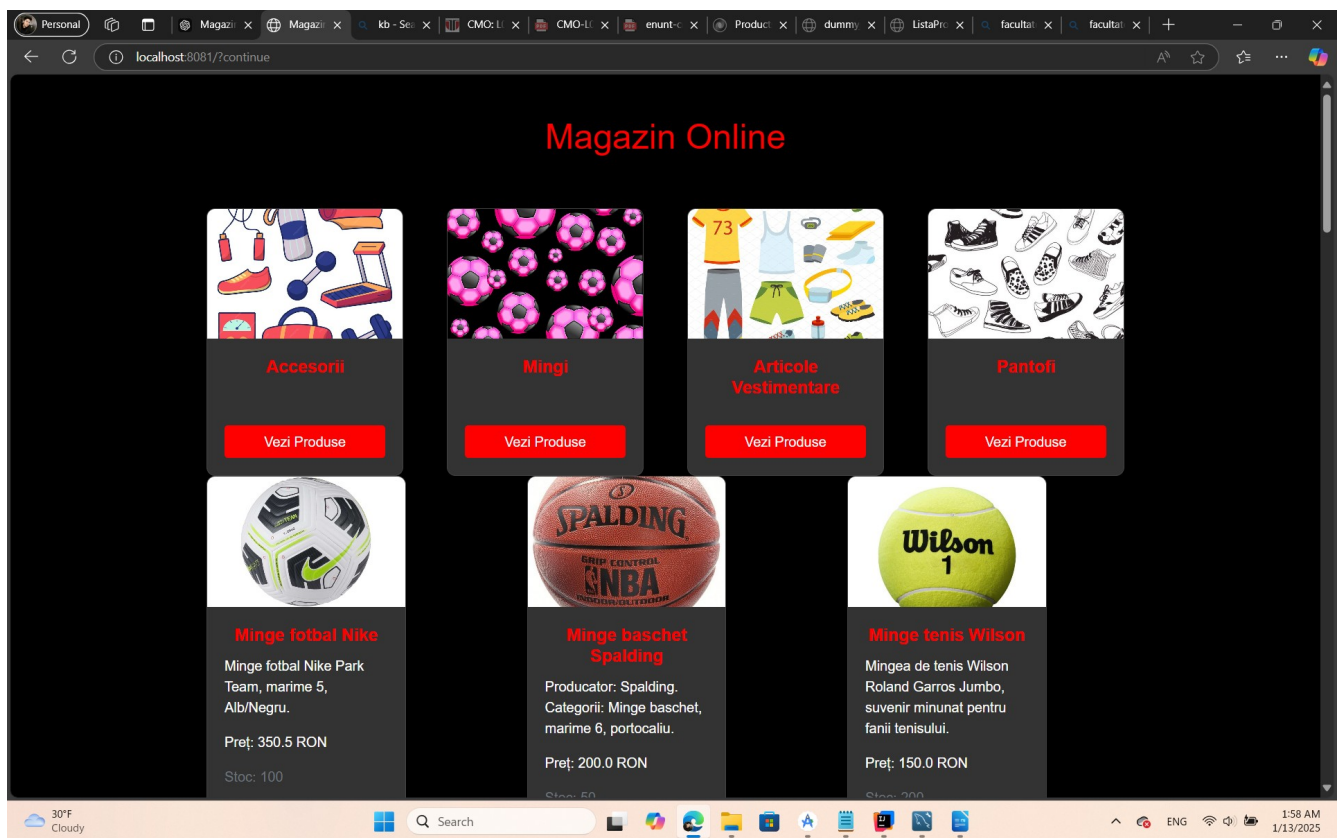
Nu am reusit sa fac logarea user ilor, doar ce realizez cu Sping Security.

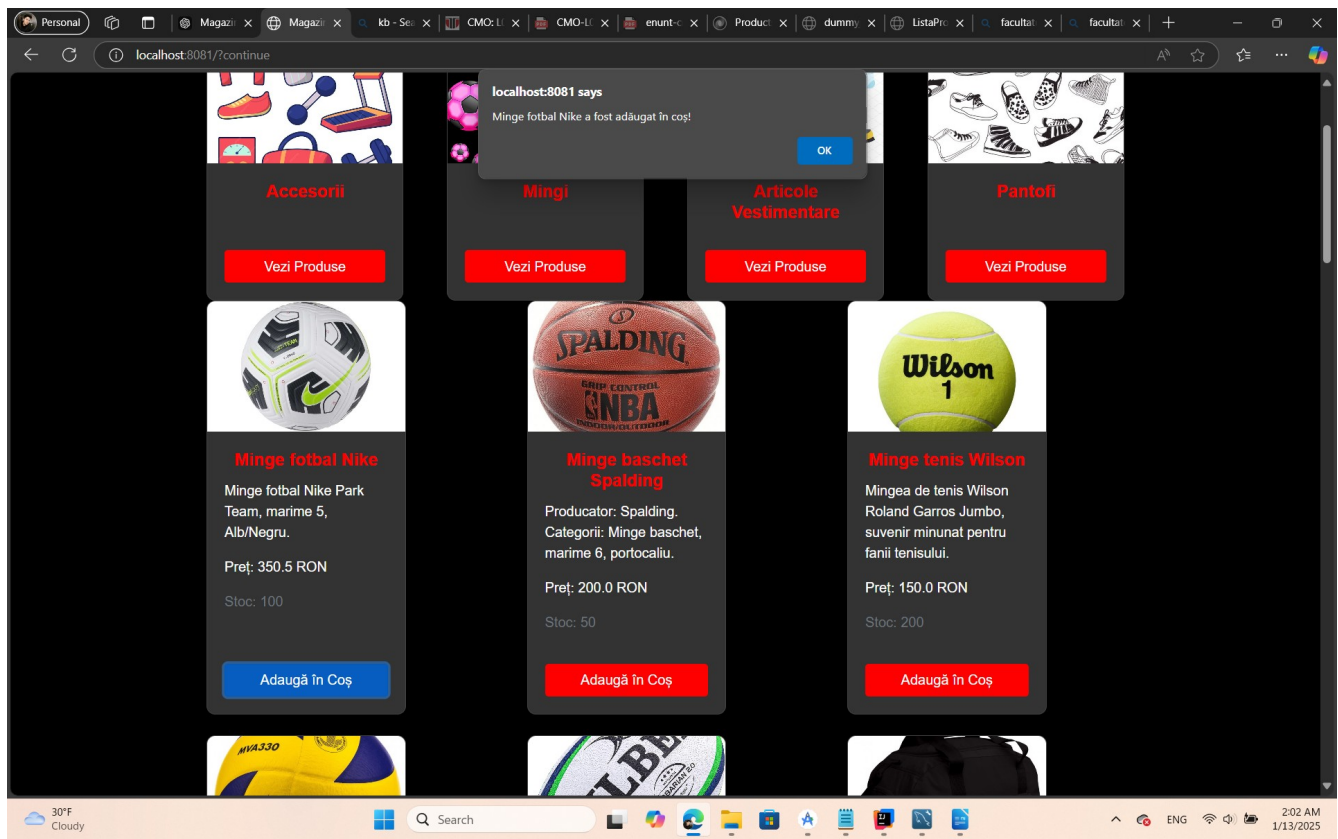
5.3. Pagina de acasă

Pagina de acasă este un punct central al aplicației **Magazin Online**, oferind utilizatorilor o privire generală asupra produselor disponibile și opțiuni pentru navigare în aplicație. Utilizatorii pot vizualiza produsele, adăuga produse în coșul de cumpărături și plasa comenzi.

Pentru această pagină, utilizatorii pot vizualiza o listă de produse, care sunt preluate din baza de date. În backend, produsele sunt interogate din baza de date folosind **Spring Data JPA**, iar în frontend, produsele sunt afișate într-o listă.

Pagina mea de acasa contine doar categoriile produselor cu buton care ar trebui sa duca spre pagina cu produsele concrete dar nu am reusit sa fac legatura, asa ca pe prima pagina am pus si produsele





Fiecare produs are un buton pentru cos iar la apasarea lui ne apare un mesaj. Din pacate nici pagina pentru vizualizarea produselor din cos nu mi functioneaza.

6. Ce funcționează / Ce nu funcționează

În această secțiune, vom descrie funcționalitățile implementate și problemele întâmpinate în dezvoltarea aplicației **Magazin Online**.

6.1. Funcționalități implementate

Aplicația **Magazin Online** utilizează **Spring Boot** pentru backend și **Thymeleaf** pentru frontend. În prezent, implementările sunt parțial complete, iar unele funcționalități sunt operaționale, deși mai sunt aspecte care necesită îmbunătățiri.

- **Pagina de logare cu Spring Security:**

- **Autentificare cu un user generat automat:** Utilizatorii pot să se autentifice folosind un **user generat automat de Spring Security** (în loc de a crea un utilizator în baza de date). Spring Security generează o parolă și un user de bază (de obicei, utilizatorul și parola pot fi vizualizate în logurile aplicației la pornire).
- **Autentificare validă:** După ce utilizatorul se autentifică, aplicația îl redirecționează către pagina principală.
- **Pagina principală:**
 - **Afișarea produselor:** Toate produsele din baza de date sunt afișate pe pagina principală. Aceste produse sunt încărcate din backend și prezentate pe frontend prin **Thymeleaf**.
 - **Categoriile de produse:** Categoriile sunt prezentate sub formă de butoane pe pagina principală. Cu toate acestea, aceste butoane nu duc momentan la pagini funcționale (deși există intenția de a le conecta la funcționalități care filtrează produsele în funcție de categorie).
 - **Butonul pentru coș:** Pe pagina principală există un buton care ar trebui să adauge produse în coș. În momentul de față, acest buton doar transmite un mesaj, dar nu interacționează efectiv cu coșul de cumpărături.

6.2. Probleme întâmpinate

În ciuda progresului în implementare, există câteva dificultăți și funcționalități nefinalizate, care afectează comportamentul complet al aplicației.

1. Legătura între paginile de produse și categoriile de produse:

- **Problema:** Butoanele de categorie sunt vizibile pe pagina principală, dar nu duc către pagini funcționale care să filtreze produsele pe baza alegerii făcute. De asemenea, nu există o legătură activă între butoanele de categorie și produsele afișate.
- **Soluție necesară:** Trebuie implementată o logică backend care să filtreze produsele în funcție de categoria aleasă și să transmită datele filtrate către pagina respectivă.

2. Gestionarea produselor în coș:

- **Problema:** Butonul pentru coș doar afișează un mesaj și nu adaugă produsele în coșul de cumpărături. Aceasta înseamnă că nu există încă o implementare completă a funcționalității coșului.
- **Soluție necesară:** Este nevoie de implementarea unei funcționalități de gestionare a coșului, unde produsele pot fi adăugate, actualizate și vizualizate în coșul de cumpărături. Ar putea fi necesar să folosești sesiuni sau o bază de date pentru a stoca produsele adăugate în coș.

3. Interacțiunea dintre backend și frontend (Thymeleaf):

- **Problema:** Legătura între frontend (Thymeleaf) și backend nu este completă în ceea ce privește adăugarea de produse în coș sau filtrarea produselor în funcție de categorie.

- **Soluție necesară:** Va trebui să implementezi metode de filtrare a produselor pe backend și să trimiți aceste date către paginile **Thymeleaf**. De asemenea, va trebui să gestionezi starea coșului de cumpărături pe server și să actualizezi pagina corespunzător.

4. Autentificarea cu utilizatori generati de Spring Security:

- **Problema:** Momentan, autentificarea se face cu un utilizator generat automat de Spring Security, iar acest lucru nu permite un control complet asupra utilizatorilor din baza de date.
- **Soluție necesară:** Ar trebui să implementezi înregistrarea unui utilizator propriu în baza de date pentru a permite crearea și gestionarea conturilor de utilizatori.

5. Îmbunătățirea navigării:

- **Problema:** Navigarea între pagini și filtrarea produselor nu sunt complet funcționale. De exemplu, butoanele de categorie nu sunt conectate la logica backend pentru a afișa produse filtrate.
- **Soluție necesară:** Trebuie adăugată logica de routing și filtrare a produselor în funcție de categorie și să se asigure că datele corecte sunt transmise între backend și frontend.

7. Surse

În această secțiune, vom detalia resursele utilizate în dezvoltarea proiectului **Magazin Online**, precum și referințele externe care au contribuit la implementarea și proiectarea aplicației.

7.1. Resurse utilizate

1. Tehnologii și framework-uri:

- **Spring Boot:** Pentru dezvoltarea backend-ului și gestionarea logicii de afaceri.
- **Spring Security:** Pentru autentificare și gestionarea securității aplicației.
- **Thymeleaf:** Motor de template utilizat pentru a integra interfața utilizatorului cu datele din backend.
- **JPA și Hibernate:** Pentru maparea obiect-relatională (ORM) și gestionarea bazei de date.
- **H2 Database:** Baza de date in-memory utilizată pentru testare și dezvoltare rapidă.
- **Maven:** Pentru gestionarea dependențelor și configurarea proiectului.
- **Bootstrap:** Pentru stilizarea și crearea unui design atractiv al interfeței utilizatorului.

2. IDE și instrumente de dezvoltare:

- **IntelliJ IDEA:** Pentru scrierea și organizarea codului.
- **Postman:** Pentru testarea API-urilor în fazele inițiale ale dezvoltării.
- **Git și GitHub:** Pentru versionarea codului și colaborare.
- **Spring Initializr:** Utilizat pentru generarea structurii de bază a proiectului.

3. Resurse proprii și exemple:

- Materialele de curs sau tutoriale utilizate pentru învățarea și implementarea tehnologiilor menționate mai sus.
- Codul scris în timpul testării și debugging-ului funcționalităților aplicației.

7.2. Referințe externe

- **Documentația oficială Spring:**
 - [Spring Boot Documentation](#)
 - [Spring Security Reference](#)
 - [Spring Data JPA Documentation](#)

Concluzie

Proiectul **Magazin Online** a reprezentat o experiență practică valoroasă, oferindu-mi ocazia să învăț și să aplic concepte fundamentale din dezvoltarea aplicațiilor web. Utilizând **Spring Boot** pentru backend și **Thymeleaf** pentru frontend, am reușit să creez o bază funcțională pentru o aplicație de tip magazin online. Deși există încă funcționalități nefinalizate, progresele realizate până acum demonstrează o înțelegere solidă a tehnologiilor utilizate.

Printre aspectele funcționale ale proiectului se numără:

- **Pagina de logare** cu Spring Security, care validează utilizatorii, deși în prezent folosește parola generată automat, fără integrarea cu baza de date.
- **Pagina principală**, care afișează produsele din baza de date, împreună cu un buton pentru "Cosul de cumpărături", care, momentan, transmite doar un mesaj.
- **Categoriile de produse** sunt afișate sub formă de butoane pe pagina principală, deși acestea nu redirecționează încă către pagini funcționale.

Deși proiectul este încă într-un stadiu intermediar, am identificat punctele în care trebuie să îmi concentrez eforturile:

1. Implementarea unei pagini de înregistrare funcționale și integrarea utilizatorilor în baza de date.
2. Corelarea claselor backend cu elementele din frontend pentru a permite navigarea între paginile aplicației.
3. Extinderea funcționalității butoanelor de categorie pentru a afișa produse specifice fiecărei categorii.
4. Adăugarea funcționalităților complete pentru coșul de cumpărături, astfel încât să permită adăugarea, ștergerea și finalizarea comenzilor.

Acest proiect m-a ajutat să aprofundez:

- Structurarea unui proiect folosind arhitectura MVC (Model-View-Controller).
- Configurarea securității aplicației web prin **Spring Security**.
- Integrarea datelor din baza de date în interfața utilizatorului prin **Thymeleaf**.

- Rezolvarea problemelor practice legate de gestionarea utilizatorilor, securitatea datelor și navigarea în aplicație.

Deși mai sunt pași de parcurs pentru a transforma această aplicație într-un magazin online complet funcțional, baza creată oferă un punct de plecare solid pentru extinderea viitoare. Am încredere că, prin rezolvarea problemelor actuale și continuarea implementării funcționalităților planificate, voi putea finaliza cu succes acest proiect.